sql上机

第一次

年份/试卷	班型	题号	难度	题目类型	为什么难
2024 上机 A 卷	未标班型	第 8 题	★★★★★	选修过全部由计算机学院开设课程的学生	“全部/所有”型查询，典型双重 `NOT EXISTS`。
2024 上机 A 卷	未标班型	第 10 题	★★★★☆	每门课程最高最终成绩学生	要算最终成绩，再找每门课最高分，还要考虑并列最高分。
2024 上机 A 卷	未标班型	第 9 题	★★★★☆ 【写错了】	人数小于 3 人，包括 0 人的学院	必须用外连接保留 0 人学院，普通内连接会漏。
2023 SQL 考试	未标班型	第 6 题	★★★★☆ 【写错了】	工号 10102 参与每类项目职责个数，未参加显示 0	外连接补 0，连接条件位置容易写错。
2023 SQL 考试	未标班型	第 10 题	★★★★☆	没有参与预算高于 160000 项目的员工	“没有参与某类项目”，适合 `NOT EXISTS`。
2021 图书库 B 卷	**普通班**	第 10 题	★★★★★	借阅过所有类别图书的学生	“所有类别都借过”，全称查询，并要求使用 `exists / not exists`。
2021 图书库 A 卷	**卓越班**	第 10 题	★★★★★	计算机技术类图书总价格排名第二小	难在并列排名，总价相同排名一致，不能直接 `limit offset`。
2021 图书库 A 卷	**卓越班**	第 9 题	★★★★☆	一天内借阅计算机技术书籍多于一本的学生	要按“学生 + 日期”分组，还要先限定类别。
2022 SQL 考试	未标班型	第 5 题	★★★★☆	没有参加“设计类”项目的男性职工	反查询，适合 `NOT EXISTS`；容易把逻辑写反。
2022 SQL 考试	未标班型	第 6 题	★★★★☆	北京工作且参加过“产品推广”项目的职员及项目个数	多表连接 + 条件筛选 + 分组计数，表关系容易连乱。

- **2023 第 6 题**：左联补 0
- **2024 第 9 题**：左联包括 0 人学院

- **2024 第 8 题**：双重 `not exists`
- **普通班第 10 题**：借阅过所有类别图书，也是双重 `not exists`

5/30 练习 排序怎么写

错误：order by desc by empid;

改正：order by empid desc;

- 正序：asc
- 倒序：desc

like和=

写法	含义
`name = '李明'`	必须完整等于“李明”
`name LIKE '李明'`	没有通配符时，效果基本等于 `=`
`name LIKE '李%'`	以“李”开头
`name LIKE '%李%'`	包含“李”
`name LIKE '_李%'`	第二个字符是“李”

实例化的意义——解决嵌套查询

e.g.

V1

select empid,empname,age,sex,depid
from employee
where exists(select * from employee natural join workson natural join project natural join category where catname='软件类' and age>35);

- 看起来似乎有道理，但是其实只会返回all或null

V2

那我们做实例化：

select empid,empname,age,sex,depid
from employee e
where exists(select * from employee e natural join workson w natural join project p natural join category c where c.catname='软件类' and e.age>35 and e.empid=w.empid);

- 但其实还是错误的！实例化起到的是连接作用，外层对e进行了实例化，内层再写一遍同名e会覆盖外层，和v1没有本质区别

V3

这个实例化是起到一个连接作用

select empid,empname,age,sex,depid
from employee e
where exists(select * from workson w natural join project p natural join category c where c.catname='软件类' and e.age>35 and e.empid=w.empid);

- 一个题外话：见到“参加过”“参与过”就要考虑exists，否则容易重复输出，因此如果不想用嵌套子查询需要在查询结果主键部分加上distinct
- trick：考试的时候依旧natural+where硬查，之后检查一下结果能否直接用distinct解决

关于group-count【一个犯傻的误会】

```
select catid,count(job)as jobNum
from worksonnaturaljoin projectnaturaljoin category
where empid='10102'
groupby catid;
```

意思是：

工号为 `10102` 的员工，在每一类项目中，担任过多少个“职责 job”。

```
elect catid,count(catid)as jobNum
from worksonnaturaljoin projectnaturaljoin category
where empid='10102'
groupby catid;
```

意思变成：

统计员工 10102 在每类项目中参与了多少条项目记录。

ANY和ALL

- any就是右面的集合里面有一个满足那个规则就行
- all是右面的集合全满足那个规则

%和_

- %代表字符串
- _代表有且只有一个

group的要求

用了 `GROUP BY` 以后，`SELECT` 后面出现的列，要么在 `GROUP BY` 里，要么被聚合函数包起来。

- 根本原因：不这样没法表示，不便于画表【分组以后，一组会被压缩成结果表里的一行】

limit

1. **单参数**

```
LIMIT n
```

表示取前 `n` 条记录。

1. **两个参数**

```
LIMIT offset, row_count
```

- `offset`：跳过多少条记录
- `row_count`：取多少条记录

示例：

```
-- 取第 1 条（最大值）
ORDERBY stuidDESC
LIMIT0,1-- 或 LIMIT 1 OFFSET 0

-- 取第 2 条（第二大）
ORDERBY stuidDESC
LIMIT1,1-- 跳过 1 条，取 1 条

-- 取第 3 条（第三大）
ORDERBY stuidDESC
LIMIT2,1-- 跳过 2 条，取 1 条

-- 取第 4 条（第四大）
ORDERBY stuidDESC
LIMIT3,1-- 跳过 3 条，取 1 条
```

6/3 练习 distinct

```
select bookid,distinct bookname, author
from borrownaturaljoin book
where stuid=200810111;
```

是错误的。

正确写法是：

```
selectdistinct bookid, bookname, author
from borrownaturaljoin book
where stuid=200810111;
```

它表示对这三个字段组成的整行去重：

```
(bookid, bookname, author)
```

`avg` 和普通属性一起显示的方法

- 2021-B-2
  select bookname,author,price,(select round(avg(price),1) from book) as avgprice
  from book
  where price>(select avg(price) from book)
  order by price DESC;

round(avg(price), 1) 四舍五入

保留一位小数

一个值得注意的问题：count(distinct majorid)

- - 3
  -- 很重要
  select degree,count(distinct majorid) as num
  from student
  group by degree;

用any/all解决最值 vs 用limit解决最值 vs minmax

找“最大/最小值”本身：优先用 MAX / MIN
找“第 x 大 / 第 x 小”：通常用 ORDER BY + LIMIT

any/all其他应用

`ANY / ALL` 更多是用来**拿子查询结果当作一条“比较线”或“比较范围”**，然后判断外层数据在不在这条线的一侧

count

`COUNT()` 常见有三种写法：

```
count(*)
count(列名)
count(distinct 列名)
```

- `count(*)`

统计**行数**，不管某一列是不是 `NULL`。

```
selectcount(*)
from student;
```

意思是：统计 `student` 表里一共有多少行学生记录。

哪怕某些学生的 `phone`、 `email` 是空的，也会被算进去。

- `count(列名)`

统计这一列中**非 NULL 的个数**。

```
selectcount(phone)
from student;
```

意思是：统计 `phone` 不为空的学生数量。

如果有 10 个学生，但其中 3 个学生 `phone` 是 `NULL`，那么：

```
count(*)=10
count(phone)=7
```

- `count(distinct 列名)`

统计这一列中**去重后、非 NULL 的个数**。

```
selectcount(distinct catid)
from book;
```

意思是：统计图书表里一共有多少种不同的类别编号。

标量子查询

select proid,projectname,(select min(budget) from project where catid='c2') as budget,50 as a
from project
where catid='c2' and budget=(select min(budget) from project where catid='c2')

proid	projectname	budget	a
p5	办公用品采购	80000	50

6/6 练习时间比较

- 越晚越大
- 需要单引号

`DATE` 比大小，本质上就是按时间先后比较：

越早的日期越小，越晚的日期越大。

比如：

```
'2024-01-01'<'2024-06-01'
```

成立，因为 1 月 1 日早于 6 月 1 日。

常见写法：

```
SELECT*
FROM borrow
WHERE borrow_date>'2024-01-01';
```

NULL

在 SQL 里判断 `NULL`，**不能用** `= NULL` **或** `<> NULL`，要用：

```
ISNULL
```

或者：

```
ISNOTNULL
```

in 和 not in

- 值 in/not in 集合
- 如果表达“集合 in/not in 集合”，需要循环
- 右值为空返回空
- NOT IN：判断“一个值不在一列值里”
  NOT EXISTS：判断“不存在满足条件的记录”

普通 `NOT IN` **左边通常是一个值，不是一个表集合。**

合法：

```
WHERE stu_idNOTIN (
SELECT stu_id
FROM takes
)
```

不合法：

```
WHERE (SELECT stu_idFROM student)NOTIN (
SELECT stu_idFROM takes
)
```

因为左边不能直接放一个多行结果。

如果你想表达：

> A 集合中的元素不在 B 集合中

应该这样写：

```
SELECT stu_id
FROM student
WHERE stu_idNOTIN (
SELECT stu_id
FROM takes
);
```

也就是：**从 A 集合里一行一行取出来，判断每个值是否不在 B 集合里。**

还有一个非常重要的坑：`NOT IN` 遇到 `NULL` 可能出问题。

例如右边子查询结果里有 `NULL`：

```
WHERE stu_idNOTIN (
SELECT stu_id
FROM takes
)
```

如果 `takes.stu_id` 里面有 `NULL`，可能导致结果为空。

起别名

有别名的表，后面必须用别名；没有别名的表，后面用原表名。

杀死比赛的关键词——WITH

```
-- 8. 给出选修过全部由计算机学院开设的课程（col_name = '计算机学院'）的学生
--  （stu_id， stu_name）（10 分）



select stu_id,stu_name
from student s
where not exists(select * from takes t natural join course c where c.co
l_id=(select col_id from college where col_name='计算机学院') and not exi
sts (select * from takes t2 where t2.stu_id=s.stu_id and t.cor_id=t2.co
r_id));



-- cs_course(课程id)
with cs_course as(
```

```
select cor_id
from course
where col_id=(select col_id from college where col_name='计算机学院')),
-- cs_num=4
cs_num as(select count(*) from cs_course),

results as(select stu_id,stu_name from student natural join takes where
cor_id in (select * from cs_course) group by stu_id having  count(*)=(s
elect * from cs_num) )

select * from results;
```

```
-- 10. 给出每门课程的课程 id、课程名和该门课程获得最高分的学生的学号、姓名，以及
-- 该学生所获分数并分数按倒序排列，其中课程最终成绩=0.3 平时成绩+0.7 期末成绩
--  (cor_id, cor_name, stu_id, stu_name,final_score)(10 分)



-- new_takes(学生id，课程id,总分)
with new_takes as(select stu_id,cor_id,mid_score*0.3+fin_score*0.7 as s
um_score from takes),
-- maxscore(课程id,最高分)
maxscore as(select cor_id,max(sum_score) as final_score from new_takes
group by cor_id),
-- maxsc_takes(课程id,学生id，相应最高分)
maxsc_takes as(select cor_id,stu_id,sum_score as final_score from new_t
akes nt where exists(select * from maxscore m where nt.cor_id=m.cor_id
and nt.sum_score=m.final_score)),
-- results(课程id，课程名字，学生id，学生名字，最终得分)
results as (select m.cor_id,c.cor_name,m.stu_id,s.stu_name,m.final_scor
e from maxsc_takes m,student s,course c where m.stu_id=s.stu_id and m.c
or_id=c.cor_id)

select * from results;
```

having

聚合函数条件一定要写到having里面

如何看报错

报错关键词	中文意思	常见原因	修改方向
`syntax error` / `SQL syntax`	语法错误	括号、逗号、关键字顺序、中文标点、最后多逗号	检查 SQL 基本结构

报错关键词	中文意思	常见原因	修改方向
`Invalid use of group function`	聚合函数用错	把 `count()`、`avg()`、`max()` 写进了 `where`	聚合条件放 `having`
`Column 'xxx' is ambiguous`	字段不明确	多个表都有同名字段	写成 `表名.字段` 或 `别名.字段`
`Unknown column 'xxx'`	找不到字段	字段名写错、表里没有、别名用错	检查字段名和别名
`Unknown table` / `Table doesn't exist`	表不存在	表名写错、临时表作用域结束、数据库没选对	检查表名、`WITH` 作用范围、当前数据库
`Duplicate column name`	重复列名	子查询或视图里多个字段同名	给字段起别名
`Every derived table must have its own alias`	子查询缺少别名	`from (select ...)` 后面没起名	给子查询加别名
`You have an error near ...`	在某处附近出错	报错点附近通常有语法问题	重点看 `near` 后面的 SQL 片段
`Operand should contain 1 column`	子查询返回列数不对	`in/select =` 后面的子查询返回了多列	子查询只保留一列
`Subquery returns more than 1 row`	子查询返回多行	`=` 后面的子查询结果不止一行	改用 `in`，或保证子查询只返回一行

你现在最常见的几个可以重点记：

你看到的报错	第一反应
`Invalid use of group function`	`count()` 之类是不是写在 `where` 里了
`ambiguous`	字段是不是没加表名/别名
`Unknown column`	字段名/别名是不是写错了
`syntax`	逗号、括号、中文标点、SQL 顺序是不是错了

SQL 正确顺序表：

顺序	子句	作用
1	`select`	选择要显示的字段
2	`from`	指定数据来自哪些表
3	`join ... on`	表之间连接
4	`where`	分组前筛选普通行
5	`group by`	按字段分组
6	`having`	分组后筛选聚合结果
7	`order by`	排序
8	`limit`	限制结果数量

order by

`order by x, y` 的意思是：

先按 x 排序；如果 x 相同，再按 y 排序。

6/9

用标量子查询规避聚合函数（聚合函数不group的时候认为整体是一个组，压缩成一行）

```
select bookname, author, price, avg(price)as avgprice
from book;
```

`avg(price)` 是聚合函数，会把整张表聚合成**一组**，所以结果容易变成**只有一行**。

但你想要的是：

```
每一本书一行 + 旁边附带全体平均价格
```

所以要把平均价当成一个**常数值**传进每一行：

```
select bookname, author, price,
       (select round(avg(price),1)from book)as avgprice
from book
where price> (select avg(price)from book)
orderby pricedesc;
```

exists1. `EXISTS ... AND EXISTS ...` 是“两次检查”

例如：

```
select*
from student s
whereexists (
select*
from borrow br
where br.stuid= s.stuid
and br.bookid='b003'
)
andexists (
select*
from borrow br
where br.stuid= s.stuid
and br.bookid='b009'
);
```

可以理解成：

```
遍历每个学生 s：
    检查一次：他有没有借过 b003
    再检查一次：他有没有借过 b009
    两个都存在，才选出来
```

所以它是：

```
外层学生循环
    第一次 exists 检查
    第二次 exists 检查
```

也就是你说的：

```
两个 exists 是并列的，两次循环/两次检查
```

它表达的是：

```
既满足条件 A，又满足条件 B
```

---

2. `NOT EXISTS (... NOT EXISTS (...))` 是“嵌套检查”

例如查“借过所有 c1 类图书的学生”：

```
select*
from student s
wherenotexists (
select*
from book b
where b.catid='c1'
andnotexists (
select*
from borrow br
where br.stuid= s.stuid
and br.bookid= b.bookid
      )
);
```

可以理解成：

```
遍历每个学生 s：
    遍历每一本 c1 类图书 b：
        检查这个学生有没有借过这本书
```

所以它是：

```
外层学生循环
    内层图书循环
        再去 borrow 表检查是否存在记录
```

它表达的是：

```
不存在一本 c1 类图书，是这个学生没借过的
```

也就是：

```
这个学生借过所有 c1 类图书
```

6/11 生死在天AVG() 是在 分组之后、HAVING 之前 算出来的

LEFT JOIN A ON B

`LEFT JOIN` 先按 `ON` 匹配；匹配不上时，右表补 `NULL`；最后 `WHERE` 再过滤整张结果表。

“右表条件”写在 `WHERE`，通常会白左联。

左表条件写在 `WHERE` 一般没问题，因为左表本来就会被保留，`WHERE` 是你主动决定“左表哪些行要留下”。